

Experiment – 8

Name: Harshit Kumawat

UID: 24BAI70025

Section: 24AIT_KRG-G1

Part A: Experimental Procedure

Instead of code, here is the logical approach and expected metrics for implementing these algorithms.

Step 1: Implementation

- **Binary Search (Divide and Conquer):** The implementation requires a sorted array. The logic involves setting a low and high pointer. It calculates the middle index, compares the middle element to the target, and then discards half of the search space based on whether the target is larger or smaller. This repeats until the target is found or the search space is exhausted.
- **Subset Sum - Decision Version (Brute Force):** This requires exploring all possible combinations of the given set. The logic typically uses recursion: for every element, the algorithm branches into two paths—one where the element is included in the current sum, and one where it is excluded. It checks if any of these branching paths eventually equal the target sum.
- **Subset Sum - Verification Version:** The implementation here is trivial. Given an array representing a proposed subset, the logic simply loops through the elements, adds them together, and checks if the final sum matches the target.
- **Traveling Salesman Problem (Brute Force):** The logic involves generating every single mathematical permutation of the cities (excluding the starting city to avoid duplicates). For each permutation, it calculates the total distance by summing the weights of the edges between consecutive cities, adding the return trip to the start, and keeping track of the minimum distance found so far.

Step 2: Measuring Metrics

If you were to run these implementations and track metrics (Execution Time, Operations, Feasibility), you would observe the following patterns:

- **Binary Search:** Operations stay in the low dozens even for millions of items. Status will always be "Completed" with near-zero execution time.

- **Subset Sum (Decision):** Operations double with every single item added to the input set. Execution time will rise sharply, and it will likely hit a "Timeout" status when the input size exceeds 25–30 items.
- **Subset Sum (Verification):** Operations equal the number of items in the subset. Execution time is virtually zero. Always "Completed."
- **TSP (Brute Force):** Operations explode factorially. It will hit a "Timeout" incredibly fast, typically failing to complete within a reasonable timeframe once the graph exceeds 12 to 15 cities.

Part B: Analysis Tasks (Critical Thinking)

1. Why does Binary Search consistently demonstrate efficient performance across large input sizes?

Binary Search operates on the principle of **Divide and Conquer**. Instead of checking elements one by one, it compares the target to the middle element of a *sorted* array and discards half of the remaining search space in a single operation.

Because the search space is halved at every step, the maximum number of comparisons required to find an element is logarithmic, denoted as $O(\log n)$. To put this into perspective: if you have an input size of 1,000,000 items, a linear search might take 1,000,000 operations. Binary Search will find the target in a maximum of just **20 operations**. Even if the dataset grows to 1 billion items, it only takes about 30 operations. This logarithmic growth curve is incredibly flat, resulting in near-instantaneous performance regardless of how massive the input becomes.

2. Explain why Subset Sum is computationally difficult to solve but relatively easy to verify.

This contrast defines the fundamental difference between the complexity classes P (Polynomial time) and NP (Nondeterministic Polynomial time).

- **Difficult to Solve:** To definitively *solve* the decision version of Subset Sum, you must, in the worst case, explore every possible combination of elements. For a set of n elements, there are 2^n possible subsets. This results in an exponential time complexity of $O(2^n)$. As the number of elements grows, 2^n explodes rapidly, making it computationally intractable for large datasets.
- **Easy to Verify:** Conversely, if someone hands you a proposed solution (a specific subset) and claims it sums to the target, you only need to add the

numbers together and check the total. If the subset has k elements, it takes k simple additions. This operates in $O(k)$ time, which is linear, highly efficient, and easy for a computer to process.

3. Identify the input size at which the Traveling Salesman Problem becomes infeasible and justify the reason.

The brute-force solution to the Traveling Salesman Problem requires generating every possible permutation of cities to find the shortest route. For n cities, this requires checking $n!$ (n -factorial) possible routes. Factorial growth is one of the most explosive growth rates in mathematics.

- At 10 cities, there are 3,628,800 routes. A modern CPU handles this in a fraction of a second.
- At 12 cities, there are 479,001,600 routes. This takes a few seconds.
- At 15 cities, there are roughly 1.3 trillion routes. This could take hours or days depending on the machine.
- At 20 cities, there are roughly 2.4 quintillion routes. A standard computer evaluating one million routes per second would take roughly **77,000 years** to complete this.

Therefore, the input size generally becomes practically infeasible on standard consumer hardware between **12 and 15 cities**.

4. Differentiate between solving a problem and verifying a given solution with appropriate examples.

- **Solving a Problem:** This means designing an algorithm that takes a raw input and computes the correct answer from scratch, without any hints.
 - *Example:* You are given the set $\{3, 34, 4, 12, 5, 2\}$ and asked, "Is there a subset that adds up to 9?" Your algorithm has to do the heavy lifting of searching combinations until it discovers that $4 + 5 = 9$.
- **Verifying a Solution:** This means you are given an input *and a certificate* (a proposed answer), and your algorithm only has to confirm if the certificate is mathematically valid.
 - *Example:* You are given the set $\{3, 34, 4, 12, 5, 2\}$, the target 9, AND the certificate $\{4, 5\}$. Your algorithm simply adds $4 + 5$, sees that it equals 9, and confirms the solution is correct.

5. Discuss why NP-Complete problems are considered the most challenging class within NP.

A problem is categorized as **NP-Complete** if it satisfies two conditions:

1. It is in NP (meaning a proposed solution can be *verified* quickly in polynomial time).
2. Every other problem in NP can be mathematically transformed (reduced) into this problem in polynomial time.

They are considered the "most challenging" because they act as the universal bottleneck for all computationally difficult problems. Because of that second condition, if you can find a way to solve *just one* NP-Complete problem (like TSP or Subset Sum) quickly, you have automatically found a way to solve *every single problem* in the entire NP class quickly.